

ML Methods for Classification Project Report

Introduction

We explore three ML algorithms for classification and clustering: K-Nearest Neighbors (KNN), K-Means clustering, and Logistic Regression.

- **K-Nearest Neighbors (KNN):** Non-parametric algorithm where predictions are made based on majority class among 'k' closest neighbors.

- **K-Means clustering:** Unsupervised learning algorithm partitioning data into 'k' distinct clusters.

- **Logistic Regression:** Classification algorithm modeling class probability using a logistic function.

These models were implemented and evaluated on our dataset to compare their effectiveness, computational efficiency, and sensitivity to hyperparameters and preprocessing techniques in order to find the best performing model for the heart disease dataset.

Methodology

Data Preparation

The dataset is made out of 13 features and 297 samples. Given the three models' sensitivity to feature scaling and irrelevant features, significant preprocessing was applied to the training data before fitting the model (for more details see figure 1, 2 and 3 in A.1). First, the data was normalized using the mean and standard deviation calculated from the training set. To mitigate the impact of potential outliers, values were then clipped based on the interquartile range (IQR). Following clipping, MinMax scaling was applied to bring all features into the [0, 1] range (all was done after searching actual field used methods). Finally, recognizing that not all features contribute equally, we employed a feature selection technique based on variance since non-variant features don't contribute to the model's performance. Only the top 10 features exhibiting the highest variance in the preprocessed training data were retained for KNN. For KMeans, we used the top 7 features after manual inspection showing a really good accuracy in the validation set as shown in **Figure 1**. See Appendix A.1 for detailed comparisons of model performance with different levels of data preprocessing. For logistic regression, we kept all features after normalization and added a bias term to each sample, as the model can automatically learn feature importance through its weights. This multi-step preprocessing aimed to standardize feature contributions, reduce noise, and improve computational efficiency and model performance.

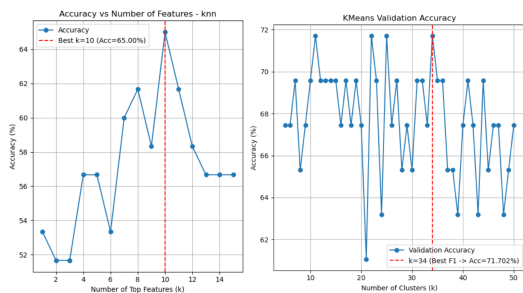


Figure 1 : Feature selection impact on KNN and KMeans
(K - the number of features retained)

Models Implementation

KNN: Our implementation uses Euclidean distance with inverse distance weighting for predictions. This weighting gives closer neighbors stronger influence, refining decision boundaries.

K-Means: We adapted this unsupervised algorithm for classification by mapping clusters to the most frequent class label among members. Centroids were initialized by randomly selecting k data points, then iteratively updated until convergence after random initialization using the seed 42 giving the best mean results over time.

Logistic Regression: Our implementation utilizes the softmax function for multi-class probability calculation and employs gradient descent for optimizing the weights. It incorporates

L2 regularization (controlled by the 'reg' hyperparameter) to prevent overfitting and assumes a bias term has been added to the input data during preprocessing, as reflected in the weight matrix dimensions.

Experiments and Results

Validation Strategy

Before hyperparameter tuning, trial and error was necessary using validation sets. We first determined the optimal training-validation split ratio to ensure reliable model evaluation. We experimented with multiple split percentages (15%, 18%, 20%, 22%, and 25%) and evaluated model performance for each configuration in the validation set. **Figures 2 and 3** shows the accuracy results across different validation split sizes for the most optimal K during hold out validation. Analysis revealed that a 20% validation split provided the best balance between having sufficient training data and a representative validation set with good performance. Since the objective was to first maximize the validation set accuracy, we chose 20% as the final validation split size. After choosing the best split size, we found that hold-out validation gave us $k = 7$ (**68.571%**) for KNN and $k = 34$ (**71.154%**) for KMeans in order to have a good balance between the two models and not overfit.

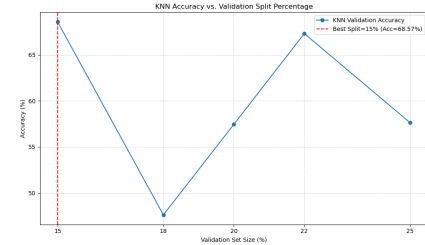


Figure 2 : KNN accuracy across validation splits for k = 7

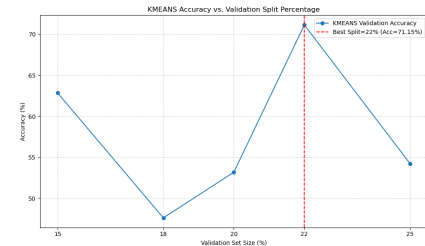


Figure 3 : K-Means accuracy across validation splits for k = 34

For logistic regression, we developed a comprehensive approach to hyperparameter optimization by analyzing first a key performance surface: mean 5 fold cross-validation accuracy using a 80-20 split. We tested learning rates ranging from 10^{-8} to 10^{-1} and maximum iterations from 125 to 3000, using a 3D surface plot to visualize the relationship between these parameters and model performance.

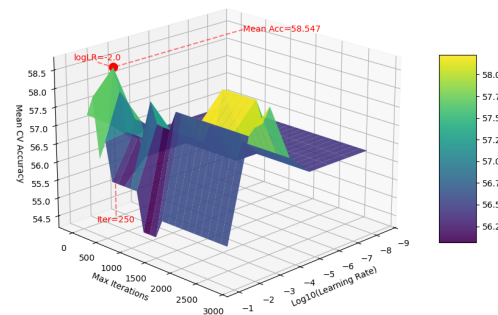


Figure 4 : Mean 5 fold cross-validation accuracy surface for validation set

With these results, we were able to first choose the optimal hyperparameters for the logistic regression model : $lr=10^{-2}$ and $iter=250$, maximizing the 5 fold CV accuracy. However, further analysis was necessary to ensure that the models weren't overfitting.

Hyperparameter Tuning

KNN: Since hold out validation gave us a good estimate of the model's performance, we were able to make a more accurate choice of k by using 5-fold stratified cross-validation to determine the optimal k as shown in **Figure 5**. Testing odd values from 1-30 (since k must be odd and must be small enough to not overfit), we found $k = 3, 5, 7$ yielded the lowest error rate. With this, we were able to match what we found in the hold-out validation and properly choose the best k . Finally, $k=7$, was chosen as the final model using the elbow method, maximizing the validation set accuracy and still giving a good performance during the training. The results are shown in **Figure 5** and proved in **Figure 6**, giving us an accuracy of **68.571%** during hold out validation.

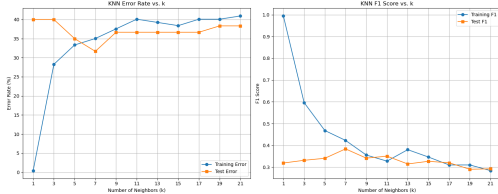


Figure 5 : KNN performance vs. neighbors (k) during cross-validation

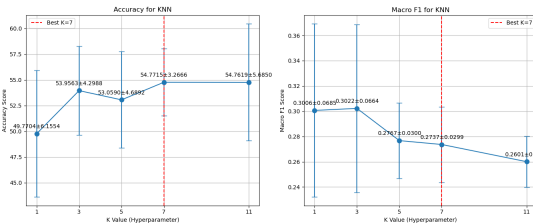


Figure 6 : Cross-validation results for KNN showing accuracy (left) and macro F1 (right) across different k values.

The vertical red line indicates the selected $k = 7$, which balances accuracy and F1 score. While accuracy remains relatively stable for $k = 5, 7$, and 11 , the macro F1 score shows a declining trend as k increases, supporting our choice of $k=7$ as optimal.

K-Means: Grid search across different cluster counts (5-50) and feature selections revealed optimal performance with $k=34$ clusters. Performance stabilized after approximately 35 clusters and started overfitting. We had first started by choosing $k = 5$ since it was the number of classes, but we quickly realized that this was not the best choice. Since K-Means clustering is unsupervised and classes may have complex, non-spherical distributions, using more clusters ($k > 5$) allows the algorithm to better capture these intra-class structures, potentially leading to improved classification performance after assigning cluster labels. Indeed, the model was not overfitting until $k=34$, giving us a performance of **71.154%**. The results are shown in **Figure 7**. For detailed F1 score analysis of K-Means across different cluster counts, see Appendix A.2.

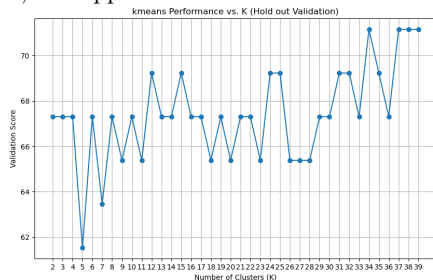


Figure 7 : K-Means accuracy vs. clusters

Logistic Regression: We developed a comprehensive approach to hyperparameter optimization by analyzing three key performance surfaces: mean cross-validation accuracy, 5 fold cross validation set accuracy, and their combined average. In the same setup as **Figure 4**, the 3D surface plots revealed that while individual metrics (hold out validation mean=59.065% at

$lr=10^{-3}$, iter=750 and 5-Fold CV accuracy=58.547% at $lr=10^{-2}$, iter=250) showed different optimal points, the combined mean accuracy surface provided a more robust solution. By maximizing this combined metric, we identified $lr=10^{-1}$ and 500 iterations as optimal parameters, ensuring good performance on both validation sets simultaneously, giving us a final accuracy for the validation set of **65.714%** and a F1 score of **0.44**.

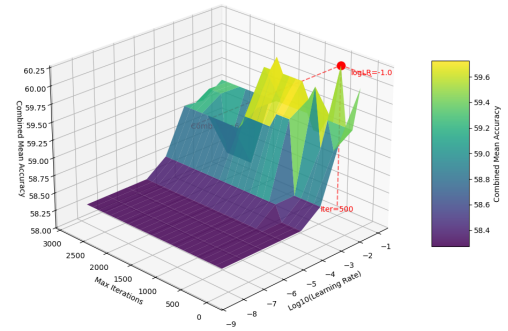


Figure 8 : Combined mean accuracy surface for validation set and 5 fold cross validation set

It shows the optimal hyperparameters (learning rate and maximum iterations) that maximize both validation set and 5 fold cross validation set performance. However, the validation set accuracy is higher than the 5 fold cross validation set accuracy, which is a bad sign that the model was overfitting during hold out validation. Thus, by taking the mean of the two accuracies, we can be confident that the model will perform well on the test set and won't overfit.

Final Model Performance

After hyperparameter tuning using validation sets and cross-validation, the final models were trained on the full preprocessed training data and evaluated on the held-out test set. The key hyperparameters and performance metrics for each model during both validation and final testing phases are summarized here.

Model	Parameters	Validation Phase		Test Phase	
		Val Acc (%)	Val F1	Test Acc (%)	Test F1
KNN	$k=7$	68.571	0.525	68.333	0.384
KMeans	$k=34$	71.154	0.357	60.000	0.277
LogReg	$lr=10^{-1}$, iter=500	65.714	0.447	63.333	0.351

Note: Val Acc/F1 are on the validation set. Test Acc/F1 are on the final test set.

Conclusion/Discussions

Our experiments highlighted the critical impact of data preprocessing; a comprehensive pipeline involving normalization, outlier clipping, MinMax scaling, and variance-based feature selection significantly boosted performance across models. However, the dataset presents significant challenges. The class distribution is notably imbalanced (training: [128, 41, 30, 30, 8], test: [32, 13, 5, 5, 5]), particularly for the least frequent class (label 4). This imbalance likely hampered the models' ability to generalize, especially for minority classes, potentially impacting metrics like the macro F1 score which would be way more important in this setup. Furthermore, the relatively small sample size (297 samples) increases sensitivity to noise and the risk of overfitting, making robust validation techniques like cross-validation essential, although not foolproof.

Future work could involve employing techniques specifically designed for imbalanced data. Exploring alternative feature selection methods or more complex models might yield further improvements, contingent on potentially acquiring more data. Ultimately, while KNN and LogReg showed promise on this dataset, their performance is intrinsically linked to the rigorous preprocessing pipeline developed and constrained by the inherent limitations of the dataset's size and balance while KMeans was not able to give us a really good performance on the test set most likely because of the predominance of the class 0.

A Appendix

A.1 Data Preprocessing Impact Analysis

The importance of thorough data preprocessing is clearly demonstrated in **Figures 1, 2, and 3**, which shows KNN performance under three different data preparation scenarios:

- **Raw data:** No preprocessing applied, resulting in poor performance due to feature scale differences and noise.
- **Partial preprocessing:** Only normalization applied without outlier handling or feature selection.
- **Full preprocessing:** Complete pipeline including normalization, outlier handling, scaling, and feature selection.

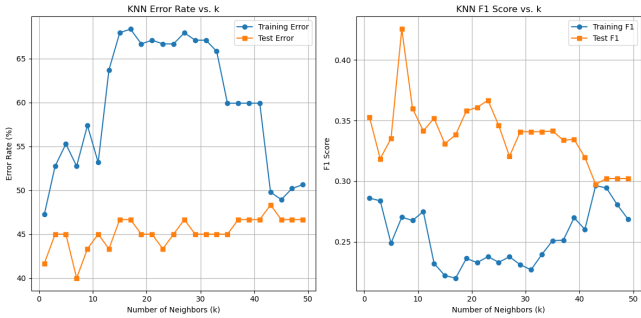


Figure 1: KNN with raw, unprocessed data

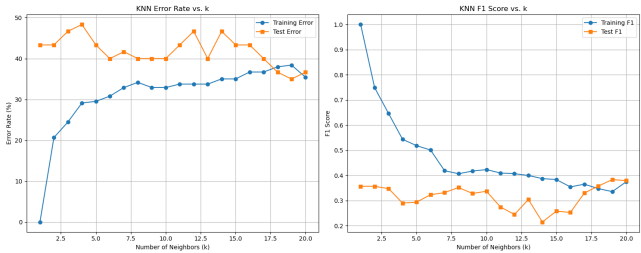


Figure 2: KNN with partial preprocessing

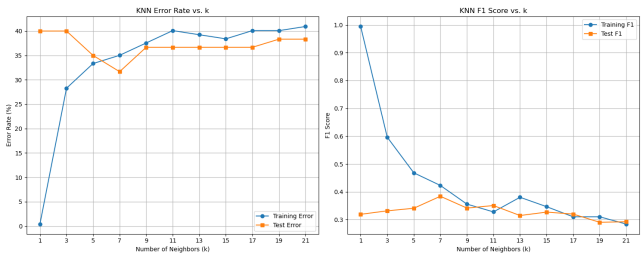


Figure 3: KNN with full preprocessing pipeline

As shown in these figures, each preprocessing step contributed to significant performance improvements. The raw data model struggled to achieve even 50% accuracy, while partial preprocessing improved performance but still fell short of optimal results. Only with the full preprocessing pipeline did the model reach its potential accuracy of almost 70%.

A.2 K-Means F1 Score Analysis

Figure 4 shows the F1 score performance of our K-Means classifier across different cluster counts. Similar to the accuracy metric shown in the main report, we observe that F1 score also stabilizes around 50 clusters. However, choosing that much clusters is not a good idea since it will overfit the model, making us choose something in the middle, while still having a good performance, i.e k=34. The F1 score is particularly important for K-Means as it measures both precision and recall, ensuring that our clusters effectively capture all instances of each class while maintaining cluster purity.

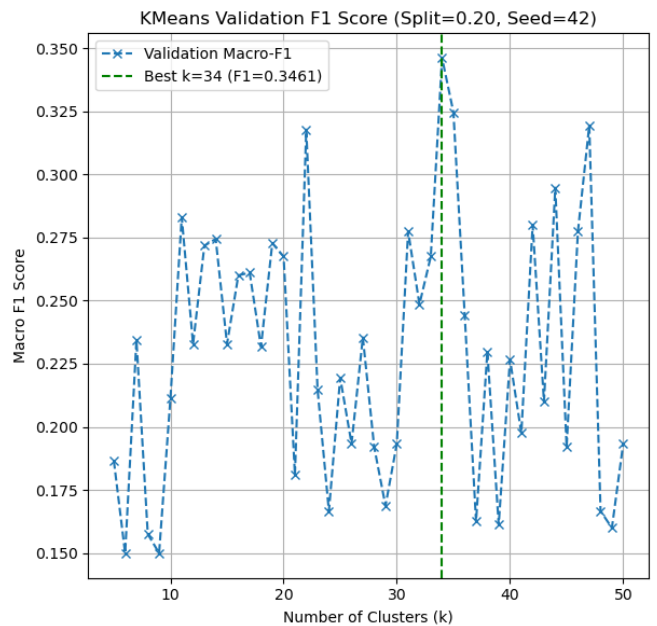


Figure 4: K-Means F1 score vs. number of clusters, showing performance stabilization around k=49